TECHNICAL DESIGN REPORT · 2026-09-28

SlideWorkflowの ローカル音声AI化

必要VRAM・システム構成・導入手順の検討

対象：Mathematica / SlideWorkflow + SourceVaultを用いたプレゼンテーションとリアルタイム質疑応答

> **結論**
> 原稿の読み上げ、発表資料に基づく質疑応答、スライド操作と割り込み・再開をローカルで実現するには、**音声認識・LLM・音声合成を分離した構成が現実的**である。30B級LLMを4bit量子化して使う場合の設計予算は**32〜48GBのVRAM**。既存のRyzen AI MAX+ 395（メインメモリ128GB）では、**GPU割当64GB**を初期目標とする。ただしGPT-Liveと同等の自然さ・全二重性・低遅延を保証するものではなく、実機評価が必要。

1. 現行システムとローカル化の範囲

SlideWorkflowはMathematicaのスライドノートブックをシナリオ駆動で生成・編集し、発表原稿（SlideTalk）と想定問答（SlideQA）を保持する。READMEでは、SourceVault経由で音声読み上げ、リアルタイム音声応答、手元リモコンを接続する構成が説明されている［1］。

SourceVaultの公開READMEでは、GPT-Live経路は音声の全二重対話、聴衆による割り込み、質疑応答後の説明再開を扱い、スライド操作と検索はクライアントのハンドラに委譲される［2］。ローカル化では、**既存のMathematica／SourceVaultの制御面を残し、クラウド音声AIの部分を置換**する。

既存要素	扱い
SlideWorkflow	スライド表示・原稿・進行制御は継続使用
SourceVault KB / Graph-RAG	スライド・図・資料の索引を事前構築し、短時間で検索
Python音声worker	ローカルASR・TTS・LLMとの接続、割り込み制御を実装
GPT-Live	ローカルLLM＋ASR＋TTS、またはOmni系モデルへ段階的に置換

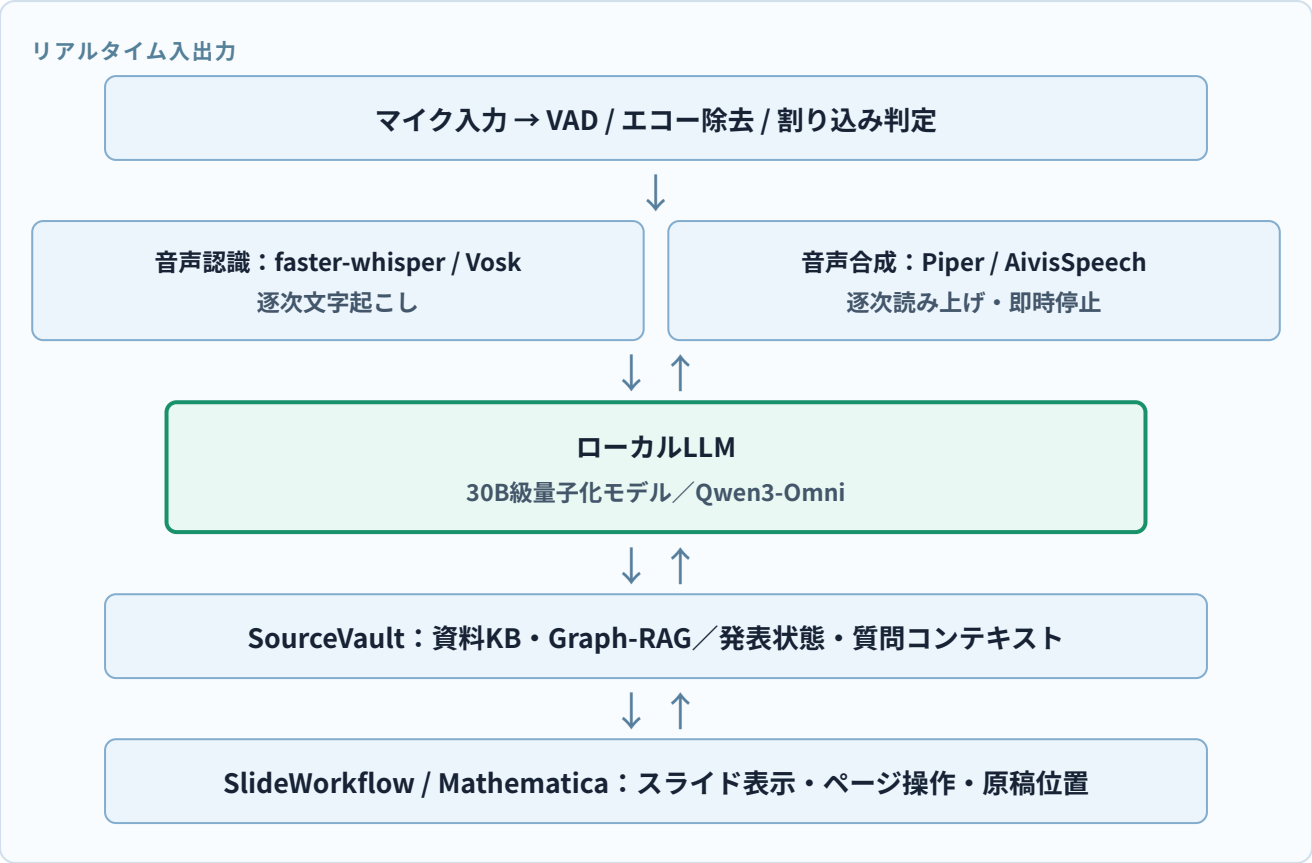
2. VRAM見積り

以下は単一の発表セッションを想定した**設計上の概算値**。モデルの量子化、音声・画像入力の長さ、コンテキスト長、GPUへのオフロード、推論エンジンによって変動する。搭載RAMと使用可能GPUメモリは区別する。

構成	VRAM予算	位置づけ
小型LLM＋ASR＋TTS	16〜24GB	発表原稿の読み上げと基本的なQ&A
30B級LLM・4bit＋ASR＋TTS	32〜48GB	複雑な質問への回答を重視。推奨初期構成
Qwen3-Omni Q4＋別TTS	32〜48GB程度	音声・画像を一体的に理解する選択肢。実装ごとのメモリ・機能検証が必要
Qwen3-Omni BF16	概ね80〜100GB以上	入力条件次第で増加。多モーダル入力・キャッシュ用の余裕が必要
大型モデル＋複数常駐モデル	96〜128GB以上	並行処理の余裕を持たせる構成

> **注意：**Qwen3-OmniのBF16は、静的なモデル重量だけでは必要メモリを判断できない。音声・映像長、Talkerの有無、推論方式で増減する。上表の数字は構築目標であり、GPU実装の互換性やリアルタイム性能を意味しない。

3. 推奨アーキテクチャ



動作シーケンス

1. 発表前に、スライド本文、話す原稿、想定問答、PDF資料・図版をSourceVaultのKBに索引化する。
2. 発表中は原稿をTTSで読み上げ、音声の再生位置と現在スライドを保存する。
3. 聴衆の発話をVADとASRで検出し、必要に応じて読み上げを停止。質問を確定する。
4. 現在のスライドと近傍資料を低遅延検索し、LLMが回答文を生成。TTSで音声応答する。
5. 質疑応答終了後に保存した進行状態へ復帰し、必要に応じて次のスライドへ進む。

全二重対話を再現する際の要件

- **エコー除去：**自分のスピーカー音を聴衆の質問と誤判定しない。
- **割り込み条件：**VADのみでなく部分認識・呼びかけ語・一定の確信度を組み合わせる。
- **応答の逐次化：**LLMの全文生成を待たずに文単位でTTSに渡す。
- **状態復元：**中断した文章の途中から再開するか、段落の先頭から再開するかを制御する。
- **操作の安全性：**スライド移動・資料検索を構造化ツール呼び出しに限定する。

4. Qwen3-Omniと分離型の選択

観点	分離型LLM	Qwen3-Omni型
音声入力	外部ASRで文字化	モデル内で音声理解可能
音声出力	独立TTS	対応ランタイムではTalker、未対応時は独立TTS
スライド画像	別のVLMが必要	画像入力を統合可能
運用	部品交換・遅延分析が容易	ランタイム機能・GPU対応に左右される
推奨用途	安定した発表と低遅延Q&A	音声・視覚情報を統合する実験系

5. Ryzen AI MAX+ 395（RAM 128GB）への導入

初期ターゲット

GPU割当64GB／30B級4bit LLM＋ローカルASR＋CPU側TTS
Windows 11上の既存MathematicaとSourceVaultを維持し、別プロセスでローカル推論サーバを起動する。メインメモリ容量は足りる見込みだが、GPU割当設定、Vulkan／ROCmドライバ、実効速度を検証する。

コンポーネント	初期実装案	検証項目
LLM	Qwen系30B級・Q4。llama.cpp系のOpenAI互換サーバ	初回応答速度、長文Q&A、ツール出力
ASR	Vosk（既存）またはfaster-whisper	日本語・英語、雑音下、部分認識
TTS	PiperまたはAivisSpeech（既存）	日本語の自然さ、文単位ストリーミング
資料検索	SourceVault KB / Graph-RAG（既存）	現在スライドに関連する資料の検索時間
対話制御	Python worker＋SourceVaultの既存ハンドラ	割り込み・再開、誤作動、安定性

導入の順序

1. **音声以外の置換：**現行GPT-Liveの質問テキストをローカルLLMへ送り、SourceVault検索とスライド操作が維持されるか確認。
2. **ASR・TTS置換：**マイク入力と読み上げをローカル化し、字幕・ログを突き合わせる。
3. **割り込みの実装：**エコー除去、質問開始・終了判定、発表位置の保存と復帰を接続。
4. **性能計測：**最初の音声応答までの時間、発話中断までの時間、VRAMピーク、GPU使用率を記録。
5. **Omni版の追加評価：**必要ならQwen3-Omniを並列の候補として試し、回答品質と応答遅延を比較。

6. 判断と留意事項

現時点では、**GPUメモリ容量よりも「発話しながら質問を聞き、即時に止まり、資料を参照して自然に回答できるか」**がGPT-Liveとの比較軸となる。分離型はGPUメモリの制御が容易である一方、全二重性を音声制御ソフトウェアで補う必要がある。Omni型は音声・視覚の統合理解に利点があるが、利用するランタイムの入出力機能を個別に確認する。

本書はシステム設計用の概算レポート。GPU割当64GBの構成での実測値、同時モデル常駐の安定性、GPT-Liveとの定量的な品質比較は含まない。

参照資料

［1］transreal, SlideWorkflow, README / user manual. github.com/transreal/SlideWorkflow

［2］transreal, SourceVault, README（SourceVault_voice, SourceVault_realtime, SourceVault_kb）。github.com/transreal/SourceVault

［3］Qwen Team, Qwen3-Omni（モデル・推論方式・メモリ要件の確認先）。huggingface.co/Qwen

［4］llama.cpp（対応モデル・音声入出力の実装状況の確認先）。github.com/ggml-org/llama.cpp